

# Repository

## Repository一覧DB Repository一覧

| Repository Interface名     | 実装名                           | 対象テーブル             | 主な責務                               |
|---------------------------|-------------------------------|--------------------|------------------------------------|
| UserRepository            | GormUserRepository            | users              | ユーザー作成、取得、認証用検索、状態更新               |
| RefreshTokenRepository    | GormRefreshTokenRepository    | refresh_tokens     | RefreshToken作成、照合、使用済み化、失効         |
| GuestSessionRepository    | GormGuestSessionRepository    | guest_sessions     | GuestSession作成、取得、最終アクセス更新         |
| BeanRepository            | GormBeanRepository            | beans              | Bean作成、更新、公開一覧、検索、詳細取得             |
| ArticleRepository         | GormArticleRepository         | articles           | Article作成、更新、公開一覧、検索、詳細取得          |
| BeanArticleRepository     | GormBeanArticleRepository     | bean_articles      | BeanとArticleの関連付け、関連記事取得           |
| RankTargetRepository      | GormRankTargetRepository      | rank_targets       | Bean / Articleを共通ランキング対象として取得      |
| ActionEventRepository     | GormActionEventRepository     | action_events      | 行動イベント記録、ランキング集計、興味集計              |
| ModalDisplayLogRepository | GormModalDisplayLogRepository | modal_display_logs | モーダル表示履歴の作成、クリック・クローズ更新            |
| ModalBlockLogRepository   | GormModalBlockLogRepository   | modal_block_logs   | モーダル非表示理由の記録                       |
| SavedItemRepository       | GormSavedItemRepository       | saved_items        | 保存、保存解除、再保存、保存一覧                   |
| RatingRepository          | GormRatingRepository          | ratings            | Good / Bad評価、再評価、評価集計              |
| ContentMetricRepository   | GormContentMetricRepository   | content_metrics    | ランキング指標の保存、ランキング取得                 |
| InterestProfileRepository | GormInterestProfileRepository | interest_profiles  | 興味スコアの保存、取得、推薦用取得                  |
| BatchRunRepository        | GormBatchRunRepository        | batch_runs         | バッチ開始、成功、失敗、履歴取得                   |
| AuditLogRepository        | GormAuditLogRepository        | audit_logs         | 監査ログ作成、検索、追跡                       |
| TxManager                 | GormTxManager                 | DB transaction     | 複数Repository更新を1つのtransactionにまとめる |

## Redis Repository一覧

| Repository Interface名      | 実装名                             | 対象    | 主な責務                              |
|----------------------------|---------------------------------|-------|-----------------------------------|
| RateLimitRepository        | RedisRateLimitRepository        | Redis | API RateLimit                     |
| EventDedupRepository       | RedisEventDedupRepository       | Redis | impression / content_view などの重複防止 |
| ModalSuppressionRepository | RedisModalSuppressionRepository | Redis | モーダル表示回数、24時間抑制                   |
| BatchLockRepository        | RedisBatchLockRepository        | Redis | バッチ二重実行防止                         |

## Repository設計ルール

| 項目               | ルール                                      |
|------------------|------------------------------------------|
| Interfaceの置き場所   | repository または usecase/repository        |
| 実装の置き場所          | repository/gorm、repository/redis         |
| usecaseの依存先      | Repository Interface、TxManager Interface |
| controllerの依存先   | usecase                                  |
| repository実装の依存先 | GORM DB、Redis Client                     |
| 返す型              | 原則 entity.*                              |
| DB Model         | GORM実装内部でのみ使う                            |
| HTTP status      | Repositoryでは扱わない                         |
| Echo Context     | Repositoryに渡さない                          |

| 項目               | ルール                                                                                                                       |
|------------------|---------------------------------------------------------------------------------------------------------------------------|
| JSON DTO         | Repositoryでは扱わない                                                                                                          |
| 業務判断             | Repositoryではない。usecaseで行う                                                                                                 |
| DB制約違反           | Repositoryで検知し、domain向けerrorへ変換する                                                                                         |
| transaction      | TxManagerで開始する。ただし、Usecaseが複数DB更新を行い、かつ途中成功すると業務状態が壊れる場合のみ使用する。単体作成、単体更新、ActionEvent作成、AuditLog作成、Redis操作、Cookie操作には使わない。 |
| GORM transaction | UsecaseにGORM DBを直接渡さない                                                                                                    |
| context.Context  | すべてのRepository Interfaceメソッドは第一引数に ctx context.Context を取る。GORM実装では db.WithContext(ctx)                                   |

## Repository Interface詳細TxManager

TxManager は、Usecase が複数DB更新を行い、途中成功すると業務状態が壊れる処理だけを1つのDB transactionとして扱うためのinterfaceである。単にRepository呼び出しが2つ以上あるだけではTx対象にしない。ActionEvent、AuditLog、Redis、Cookie、メール送信、外部APIはDB transactionに含めない。

Usecase は GORM の `*gorm.DB` を直接扱わない。

Usecase は `TxManager` interface を通して transaction を開始し、transaction 内では `TxRepos` 経由で Repository を呼び出す。

GORM の transaction 開始・commit・rollback は `GormTxManager` が担当。

## TxRepos一覧

| Interface | Method                     | 戻り値                                  | 用途                                                   |
|-----------|----------------------------|--------------------------------------|------------------------------------------------------|
| TxRepos   | <code>User</code>          | <code>UserRepository</code>          | Refresh reuse検知、LogoutAllDevicesでtoken_versionを更新する  |
| TxRepos   | <code>RefreshToken</code>  | <code>RefreshTokenRepository</code>  | Refresh rotation、reuse検知、全端末logoutでRefreshTokenを更新する |
| TxRepos   | <code>Bean</code>          | <code>BeanRepository</code>          | Bean公開/非公開時に公開状態を更新する                                |
| TxRepos   | <code>Article</code>       | <code>ArticleRepository</code>       | Article公開/非公開時に公開状態を更新する                             |
| TxRepos   | <code>RankTarget</code>    | <code>RankTargetRepository</code>    | 公開/非公開時にランキング対象状態を同期する                               |
| TxRepos   | <code>BeanArticle</code>   | <code>BeanArticleRepository</code>   | 関連Articleの一括差し替え・並び替えを行う                             |
| TxRepos   | <code>ContentMetric</code> | <code>ContentMetricRepository</code> | RankingBatch成功時にmetricsを保存する                         |
| TxRepos   | <code>BatchRun</code>      | <code>BatchRunRepository</code>      | RankingBatch成功時に成功状態を保存する                            |

## GORM実装で行うこと

| 処理            | 内容                                                                                                                       |
|---------------|--------------------------------------------------------------------------------------------------------------------------|
| transaction開始 | <code>db.WithContext(ctx).Transaction(...)</code> を使う                                                                    |
| txの受け渡し       | <code>context.Context</code> に transaction 用DBを保存しない。transaction内でTx用Repository群を生成し、 <code>TxRepos</code> としてUsecaseへ渡す |
| 成功時           | <code>fn</code> が <code>nil</code> を返した場合のみ commit する                                                                    |
| 失敗時           | <code>fn</code> が <code>error</code> を返した場合は rollback する                                                                 |
| panic時        | rollbackしたうえでpanicを再送付、または共通errorへ変換する                                                                                   |
| Redis         | DB transactionには含めない                                                                                                     |
| Cookie        | DB transactionには含めない。Controllerで設定・削除する                                                                                  |
| メール送信         | DB transactionには含めない                                                                                                     |
| 外部API         | DB transactionには含めない                                                                                                     |

## Interface定義

```

type TxManager interface {
    WithinTx(ctx context.Context, fn func(ctx context.Context, tx TxRepos) error) error
}

type TxRepos interface {
    User() UserRepository
    RefreshToken() RefreshTokenRepository

    Bean() BeanRepository
    Article() ArticleRepository
    RankTarget() RankTargetRepository
    BeanArticle() BeanArticleRepository

    ContentMetric() ContentMetricRepository
    BatchRun() BatchRunRepository
}

```

## TxManagerの方針

| 項目              | 方針                                                   |
|-----------------|------------------------------------------------------|
| transaction開始位置 | Usecaseが TxManager.WithinTx() を呼び出す                  |
| transaction実装   | Infrastructure / GORM実装側の GormTxManager が担当する        |
| Usecaseが扱うもの    | TxManager interface と TxRepos interface              |
| Usecaseが扱わないもの  | *gorm.DB、GORM transaction、Redis Client、Cookie        |
| Repositoryの責務   | 単体のDB操作を行う。transactionの開始・commit・rollbackは行わない       |
| Controllerの責務   | Usecase呼び出し、HTTP status変換、Cookie設定・削除                |
| ctxの扱い          | context.Context にTx用DBを保存しない                         |
| Redisの扱い        | Redis RepositoryはDB transactionには含めない                |
| Cookieの扱い       | Cookie設定・削除はDB commit後、またはUsecase成功後にControllerで行う   |
| ActionEventの扱い  | 保存・評価・モーダル操作の副作用として作るActionEventはTx外でbest effort作成する |
| AuditLogの扱い     | AuditLog.Createは原則Tx外でbest effort作成する                |
| rollback条件      | Tx内に残したRepository error、Tx内に残した業務error、panic         |
| commit条件        | Tx内の全処理が成功し、fnがnilを返した場合のみ                           |
| Txを使う条件         | 複数DB更新の途中成功により、認証状態・公開状態・ランキング状態・一括差し替え状態が壊れる場合のみ    |

## Usecaseからの呼び出し

| Usecase                                       | Tx内で行う処理                                                                                    | Tx外で行う処理                                                           |
|-----------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| <code>AuthUsecase.Login</code>                | なし                                                                                          | RefreshToken.Create、<br>AuditLog.Create(best effort)、Co<br>定       |
| <code>AuthUsecase.Refresh</code>              | RefreshToken.FindByTokenHashWithUserForUpdate、<br>RefreshToken.Create、RefreshToken.MarkUsed | AuditLog.Create(best effort)、Co<br>定                               |
| <code>AuthUsecase.RefreshReuseDetected</code> | RefreshToken.RevokeFamily、<br>User.IncrementTokenVersion                                    | AuditLog.Create(best effort)、Co<br>除                               |
| <code>AuthUsecase.LogoutCurrentFamily</code>  | なし                                                                                          | RefreshToken.RevokeFamily、<br>AuditLog.Create(best effort)、Co<br>除 |

| Usecase                              | Tx内で行う処理                                                                    | Tx外で行う処理                                                                                                 |
|--------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| AuthUsecase.LogoutAllDevices         | RefreshToken.RevokeByUserID、<br>User.IncrementTokenVersion                  | AuditLog.Create(best effort)、Co<br>除                                                                     |
| SavedItemUsecase.Save                | なし                                                                          | RankTarget.ExistsActiveByID、<br>SavedItem.SaveOrRestore、<br>ActionEvent.Create(save / best ef            |
| SavedItemUsecase.Remove              | なし                                                                          | SavedItem.Remove                                                                                         |
| RatingUsecase.Rate                   | なし                                                                          | RankTarget.ExistsActiveByID、<br>Rating.Upsert、<br>ActionEvent.Create(rating / best e                     |
| ModalUsecase.ShowModal               | なし                                                                          | ModalDisplayLog.Create、<br>ActionEvent.Create(modal_impres<br>best effort)、<br>ModalSuppression.SetShown |
| ModalUsecase.ClickModal              | なし                                                                          | ModalDisplayLog.MarkClickedFor<br>ActionEvent.Create(modal_click /<br>effort)                            |
| ModalUsecase.CloseModal              | なし                                                                          | ModalDisplayLog.MarkClosedFor.<br>ActionEvent.Create(modal_close ,<br>effort)、ModalSuppression.SetCl     |
| AdminBeanUsecase.CreateBean          | なし                                                                          | Bean.Create、AuditLog.Create(be<br>effort)                                                                |
| AdminBeanUsecase.UpdateBean          | なし                                                                          | Bean.Update、AuditLog.Create(b<br>effort)                                                                 |
| AdminBeanUsecase.PublishBean         | Bean.UpdatePublished、RankTarget.FindOrCreate、<br>RankTarget.UpdateActive    | AuditLog.Create(best effort)                                                                             |
| AdminBeanUsecase.UnpublishBean       | Bean.UpdatePublished、RankTarget.UpdateActive                                | AuditLog.Create(best effort)                                                                             |
| AdminArticleUsecase.CreateArticle    | なし                                                                          | Article.Create、AuditLog.Create(t<br>effort)                                                              |
| AdminArticleUsecase.UpdateArticle    | なし                                                                          | Article.Update、AuditLog.Create(<br>effort)                                                               |
| AdminArticleUsecase.PublishArticle   | Article.UpdatePublished、RankTarget.FindOrCreate、<br>RankTarget.UpdateActive | AuditLog.Create(best effort)                                                                             |
| AdminArticleUsecase.UnpublishArticle | Article.UpdatePublished、RankTarget.UpdateActive                             | AuditLog.Create(best effort)                                                                             |
| AdminRelationUsecase.CreateRelation  | なし                                                                          | Bean.ExistsByID、Article.ExistsBy<br>BeanArticle.Create、<br>AuditLog.Create(best effort)                  |
| AdminRelationUsecase.DeleteRelation  | なし                                                                          | BeanArticle.Delete、<br>AuditLog.Create(best effort)                                                      |

| Usecase                                              | Tx内で行う処理                                      | Tx外で行う処理                                                                                                     |
|------------------------------------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>AdminRelationUsecase.ReplaceRelations</code>   | BeanArticle.ReplaceByBeanID                   | Bean.ExistsById、Article.ExistsBy<br>AuditLog.Create(best effort)                                             |
| <code>AdminRelationUsecase.UpdateDisplayOrder</code> | BeanArticle.ReplaceByBeanID                   | Bean.ExistsById、Article.ExistsBy<br>AuditLog.Create(best effort)                                             |
| <code>RankingBatchUsecase.Recalculate</code>         | ContentMetric.BulkUpsert、BatchRun.MarkSuccess | BatchLock.Acquire、<br>BatchLock.Release、<br>BatchRun.MarkFailed、<br>AuditLog.Create(best effort)             |
| <code>InterestBatchUsecase.Recalculate</code>        | なし                                            | InterestProfile.BulkUpsert、<br>BatchRun.MarkSuccess、<br>BatchRun.MarkFailed、<br>AuditLog.Create(best effort) |
| <code>CleanupUsecase.DeleteExpired</code>            | なし                                            | RefreshToken.DeleteExpired、<br>GuestSession.DeleteExpired、<br>InterestProfile.DeleteExpired                  |

## UserRepository

| メソッド                               | 引数                                                       | 戻り値                                | 用途                                           |
|------------------------------------|----------------------------------------------------------|------------------------------------|----------------------------------------------|
| <code>Create</code>                | <code>user *entity.User</code>                           | <code>error</code>                 | signup時にUserを作成する                            |
| <code>FindByID</code>              | <code>id uint64</code>                                   | <code>*entity.User, error</code>   | userIDからUserを取得する                            |
| <code>FindByEmail</code>           | <code>email string</code>                                | <code>*entity.User, error</code>   | login時にemailでUserを取得する                       |
| <code>ExistsByEmail</code>         | <code>email string</code>                                | <code>bool, error</code>           | signup前にemail重複を確認する                         |
| <code>UpdateTokenVersion</code>    | <code>userID uint64, tokenVersion int</code>             | <code>error</code>                 | logout、不正検知、全端末logoutで<br>token_versionを更新する |
| <code>IncrementTokenVersion</code> | <code>userID uint64</code>                               | <code>error</code>                 | token_versionを1増やす                           |
| <code>UpdateStatus</code>          | <code>userID uint64, status<br/>entity.UserStatus</code> | <code>error</code>                 | AdminがUser状態を変更する                            |
| <code>List</code>                  | <code>limit int, offset int</code>                       | <code>[]*entity.User, error</code> | Admin画面でUser一覧を取得する                          |

## GORM実装で使う取得条件

| メソッド                               | GORM取得条件                                                               | 何をするのか                                              |
|------------------------------------|------------------------------------------------------------------------|-----------------------------------------------------|
| <code>FindByID</code>              | <code>WHERE id = ?</code>                                              | 指定されたuserIDに一致するUserを1件取得する                         |
| <code>FindByEmail</code>           | <code>WHERE email = ?</code>                                           | login時に、入力されたemailに一致するUserを取得する                    |
| <code>ExistsByEmail</code>         | <code>WHERE email = ? の COUNT</code>                                   | signup前に、同じemailのUserが既に存在しないか確認する                  |
| <code>UpdateTokenVersion</code>    | <code>WHERE id = ?</code>                                              | 指定Userのtoken_versionを指定値に更新する                       |
| <code>IncrementTokenVersion</code> | <code>UPDATE token_version = token_version + 1 WHERE<br/>id = ?</code> | 既存AccessTokenを強制失効するため、token_version<br>を1増やす       |
| <code>UpdateStatus</code>          | <code>WHERE id = ?</code>                                              | AdminがUserの状態をactive / suspended / deletedへ<br>変更する |
| <code>List</code>                  | <code>ORDER BY id DESC LIMIT ? OFFSET ?</code>                         | Admin画面でUser一覧を新しい順に取得する                            |

## usecaseからの呼び出し

| usecase                         | 呼び出し                                                                  | 何をするのか                             |
|---------------------------------|-----------------------------------------------------------------------|------------------------------------|
| <code>AuthUsecase.Signup</code> | <code>UserRepository.ExistsByEmail →<br/>UserRepository.Create</code> | 新規ユーザーを作成する。RefreshTokenは<br>発行しない |

|                                  |                                                                                      |                                                                        |
|----------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| AuthUsecase.Login                | UserRepository.FindByEmail → RefreshTokenRepository.Create                           | ログイン成功時にRefreshTokenを発行する                                              |
| AuthUsecase.Refresh              | FindByID                                                                             | AccessToken再発行前にUserが存在するか確認する                                         |
| AuthUsecase.LogoutCurrentFamily  | RefreshTokenRepository.FindByTokenHashWithUser → RefreshTokenRepository.RevokeFamily | 現在のRefreshToken familyのみ失効する。UserRepository.IncrementTokenVersionは呼ばない |
| AuthUsecase.LogoutAllDevices     | RefreshTokenRepository.RevokeByUserID → UserRepository.IncrementTokenVersion         | ユーザーの全RefreshTokenを失効し、既存AccessTokenも無効化する                             |
| AuthUsecase.RefreshReuseDetected | RefreshTokenRepository.RevokeFamily → UserRepository.IncrementTokenVersion           | RefreshToken再利用検知時にfamily失効とAccessToken無効化を行う                          |
| AdminUserUsecase.SuspendUser     | UpdateStatus                                                                         | Adminが対象Userを停止状態にする                                                   |
| AdminUserUsecase.ListUsers       | List                                                                                 | Admin画面でUser一覧を取得する                                                    |

## RefreshTokenRepository

| メソッド                             | 引数                                                    | 戻り値                         | 用途                                       |
|----------------------------------|-------------------------------------------------------|-----------------------------|------------------------------------------|
| FindByTokenHash                  | tokenHash string                                      | *entity.RefreshToken, error | refresh時にtoken_hashで取得する                 |
| FindByTokenHashWithUser          | tokenHash string                                      | *entity.RefreshToken, error | UserもPreloadして取得する                       |
| FindByTokenHashWithUserForUpdate | tokenHash string                                      | *entity.RefreshToken, error | refresh rotation時に対象行をlockして取得する         |
| MarkUsed                         | id uint64, usedAt time.Time, replacedByTokenID uint64 | error                       | 古いRefreshTokenを使用済みにし、置き換え後token IDを保存する |
| Create                           | token *entity.RefreshToken                            | error                       | RefreshTokenを作成する                        |
| Revoke                           | id uint64, revokedAt time.Time                        | error                       | token単体を失効する                             |
| RevokeFamily                     | familyID string, revokedAt time.Time                  | error                       | reuse検知時やlogout時に同familyをまとめて失効する        |
| RevokeByUserID                   | userID uint64, revokedAt time.Time                    | error                       | 全端末logout時にUserの全refresh tokenを失効する      |
| DeleteExpired                    | now time.Time                                         | int64, error                | 期限切れtokenを削除する                           |

| 注意          | 内容                                                      |
|-------------|---------------------------------------------------------|
| 更新条件        | WHERE id = ? AND used_at IS NULL AND revoked_at IS NULL |
| 更新件数0件      | reuse検知または競合として扱う                                       |
| transaction | 新token作成と同一transactionで行う                               |

## GORM実装で使う取得条件

| メソッド                    | GORM取得条件                                                | 何をするのか                                                          |
|-------------------------|---------------------------------------------------------|-----------------------------------------------------------------|
| FindByTokenHash         | WHERE token_hash = ?                                    | Cookieで送られてきたRefreshTokenをhash化し、DBに同じhashがあるか探す                |
| FindByTokenHashWithUser | Preload("User").Where("token_hash = ?", tokenHash)      | RefreshTokenを探しつつ、そのtokenを持つUser情報も一緒に取得する                      |
| MarkUsed                | WHERE id = ? AND used_at IS NULL AND revoked_at IS NULL | 古いRefreshTokenにused_atとreplaced_by_token_idを入れる。失効済みtokenは更新しない |
| Revoke                  | WHERE id = ? AND revoked_at IS NULL                     | RefreshToken 1件にrevoked_atを入れて失効させる                             |
| RevokeFamily            | WHERE family_id = ? AND revoked_at IS NULL              | 同じfamily_idのRefreshTokenをまとめて失効させる                              |
| RevokeByUserID          | WHERE user_id = ? AND revoked_at IS NULL                | 指定UserのRefreshTokenをまとめて失効させる                                   |
| DeleteExpired           | WHERE expires_at < ?                                    | 有効期限切れのRefreshTokenをDBから削除する                                    |

## usecaseからの呼び出し

| usecase                                         | 呼び出し                                                                         | 何をするのか                                                                         |
|-------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>AuthUsecase.Login</code>                  | Create                                                                       | ログイン成功時にRefreshTokenを発行し、hash化してDBに保存する。Txは使わない                                |
| <code>AuthUsecase.Refresh</code>                | TxManager.WithinTx内で<br>FindByTokenHashWithUserForUpdate → Create → MarkUsed | 古いRefreshTokenを行ロック付きで取得し、新RefreshToken作成と旧RefreshToken使用済み化を同じTxで行う           |
| <code>AuthUsecase.RefreshReuseDetected</code>   | TxManager.WithinTx内で RevokeFamily → UserRepository.IncrementTokenVersion     | 使用済みRefreshTokenが再利用された場合、同じfamilyを失効し、既存AccessTokenも強制失効する                    |
| <code>AuthUsecase.LogoutCurrentToken</code>     | FindByTokenHash → Revoke                                                     | 現在のRefreshTokenだけを失効する。Txは使わない                                                 |
| <code>AuthUsecase.LogoutCurrentFamily</code>    | FindByTokenHash → RevokeFamily                                               | 現在のRefreshToken familyのみ失効する。UserRepository.IncrementTokenVersionは呼ばない。Txは使わない |
| <code>AuthUsecase.LogoutAllDevices</code>       | TxManager.WithinTx内で RevokeByUserID → UserRepository.IncrementTokenVersion   | Userの全RefreshTokenと既存AccessTokenを失効する                                          |
| <code>CleanupUsecase.DeleteExpiredTokens</code> | DeleteExpired                                                                | 有効期限切れRefreshTokenを削除する。Txは使わない                                                |

## GuestSessionRepository

| メソッド                              | 引数                                                                | 戻り値                                      | 用途                             |
|-----------------------------------|-------------------------------------------------------------------|------------------------------------------|--------------------------------|
| <code>Create</code>               | <code>session *entity.GuestSession</code>                         | <code>error</code>                       | GuestSessionを新規作成する            |
| <code>FindByID</code>             | <code>id uint64</code>                                            | <code>*entity.GuestSession, error</code> | IDでGuestSessionを取得する           |
| <code>FindBySessionKeyHash</code> | <code>hash string</code>                                          | <code>*entity.GuestSession, error</code> | CookieのhashからGuestSessionを取得する |
| <code>Touch</code>                | <code>id uint64, lastSeenAt time.Time, expiresAt time.Time</code> | <code>error</code>                       | 最終アクセス日時と期限を更新する               |
| <code>DeleteExpired</code>        | <code>now time.Time</code>                                        | <code>int64, error</code>                | 期限切れGuestSessionを削除する          |

## GORM実装で使う取得条件

| メソッド                              | GORM取得条件                                | 何をするのか                                              |
|-----------------------------------|-----------------------------------------|-----------------------------------------------------|
| <code>FindByID</code>             | <code>WHERE id = ?</code>               | guest_session_idからGuestSessionを取得する                 |
| <code>FindBySessionKeyHash</code> | <code>WHERE session_key_hash = ?</code> | Cookieのguest session keyをhash化し、DB上のGuestSessionを探す |
| <code>Touch</code>                | <code>WHERE id = ?</code>               | 最終アクセス日時と有効期限を更新する                                  |
| <code>DeleteExpired</code>        | <code>WHERE expires_at &lt; ?</code>    | 有効期限切れのGuestSessionを削除する                            |

## usecaseからの呼び出し

| usecase                                                | 呼び出し                                                                                     | 何をするのか                                         |
|--------------------------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------|
| <code>GuestSessionUsecase.GetOrCreate</code>           | <code>FindBySessionKeyHash</code> → なければ<br><code>Create</code> → あれば <code>Touch</code> | Guestを識別するsessionを取得し、なければ作成し、あれば最終アクセス日時を更新する |
| <code>EventUsecase.RecordEvent</code>                  | <code>FindByID</code>                                                                    | Guest行動イベントを記録する前にGuestSessionの存在を確認する         |
| <code>InterestUsecase.RecalculateGuest</code>          | <code>FindByID</code>                                                                    | Guestの興味スコア再計算前にGuestSessionを確認する              |
| <code>CleanupUsecase.DeleteExpiredGuestSessions</code> | <code>DeleteExpired</code>                                                               | 期限切れGuestSessionを削除する                          |

## BeanRepository

| メソッド | 引数 | 戻り値 | 用途 |
|------|----|-----|----|
|------|----|-----|----|

|                     |                             |                       |                          |
|---------------------|-----------------------------|-----------------------|--------------------------|
| Create              | bean *entity.Bean           | error                 | AdminがBeanを作成する          |
| Update              | bean *entity.Bean           | error                 | AdminがBeanを更新する          |
| FindByID            | id uint64                   | *entity.Bean, error   | Bean詳細取得                 |
| FindPublishedByID   | id uint64                   | *entity.Bean, error   | 公開中Bean詳細取得              |
| ListPublished       | limit int, offset int       | []*entity.Bean, error | 公開中Bean一覧取得              |
| SearchPublished     | filter BeanSearchFilter     | []*entity.Bean, error | Bean検索                   |
| FindByIDs           | ids []uint64                | []*entity.Bean, error | ranking結果の実体取得           |
| ExistsByID          | id uint64                   | bool, error           | Admin・RankTarget作成時の存在確認 |
| ExistsPublishedByID | id uint64                   | bool, error           | 公開中Beanの存在確認             |
| UpdatePublished     | id uint64, isPublished bool | error                 | 公開状態を変更する                |

## GORM実装で使う取得条件

| メソッド                | GORM取得条件                                                    | 何をするのか                               |
|---------------------|-------------------------------------------------------------|--------------------------------------|
| FindByID            | WHERE id = ?                                                | Admin用に、公開状態に関係なくBeanを1件取得する         |
| FindPublishedByID   | WHERE id = ? AND is_published = true                        | 一般ユーザー向けに、公開中のBeanだけ取得する             |
| ListPublished       | WHERE is_published = true ORDER BY id DESC LIMIT ? OFFSET ? | 公開中Beanを新しい順に一覧取得する                  |
| SearchPublished     | WHERE is_published = true にfilter条件を追加                      | 公開中Beanを検索条件で絞り込む                    |
| FindByIDs           | WHERE id IN ?                                               | ランキングやモーダル候補の結果から、複数Beanの実体を取得する     |
| ExistsByID          | WHERE id = ? の COUNT                                        | Admin操作やRankTarget作成前にBeanが存在するか確認する |
| ExistsPublishedByID | WHERE id = ? AND is_published = true の COUNT                | 一般ユーザー操作前に、公開中Beanとして存在するか確認する       |
| UpdatePublished     | WHERE id = ?                                                | Beanの公開状態を変更する                       |

## usecaseからの呼び出し

| usecase                        | 呼び出し                                                                                                                                       | 何をするのか                                                                           |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| AdminBeanUsecase.CreateBean    | Create → AuditLogRepository.Create                                                                                                         | Beanを作成する。作成時点ではRankTargetを作成しない。AuditLog作成に失敗してもBean作成はrollbackしない              |
| AdminBeanUsecase.UpdateBean    | FindByID → Update → AuditLogRepository.Create                                                                                              | 更新対象のBeanを確認してから更新する。AuditLog作成に失敗してもBean更新はrollbackしない                          |
| AdminBeanUsecase.PublishBean   | TxManager.WithinTx内で UpdatePublished → RankTargetRepository.FindOrCreate → RankTargetRepository.UpdateActive。その後 AuditLogRepository.Create | Beanの公開状態とランキング対象の有効状態を同期する。AuditLog作成はTx外で行い、AuditLog作成に失敗しても公開処理本体はrollbackしない |
| AdminBeanUsecase.UnpublishBean | TxManager.WithinTx内で UpdatePublished → RankTargetRepository.UpdateActive。その後 AuditLogRepository.Create                                     | Beanの非公開状態とランキング対象の無効状態を同期する。AuditLog作成に失敗しても非公開処理本体はrollbackしない                 |
| BeanUsecase.GetBeanDetail      | FindPublishedByID                                                                                                                          | 公開中Beanの詳細を取得する                                                                  |
| BeanUsecase.ListBeans          | ListPublished                                                                                                                              | 公開中Bean一覧を取得する                                                                   |
| SearchUsecase.SearchBeans      | SearchPublished                                                                                                                            | Bean検索結果を返す。ActionEventRepositoryは直接呼ばない                                         |
| RankingUsecase.ListBeanRanking | ContentMetricRepository.ListRanking → RankTargetRepository → FindByIDs                                                                     | ランキング指標からBeanのIDを取り出し、Bean本体を取得する                                                |
| ModalUsecase.GetCandidates     | FindByIDs                                                                                                                                  | モーダル候補のRankTargetからBean本体を取得する                                                   |

## SearchPublished の sort 方針



| sort    | GORM取得条件                                                                                                                      | 説明                                 |
|---------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| score   | rank_targets と content_metrics をLEFT JOINし、content_metrics.score DESC, id DESC                                                | ランキングスコア順。metricsがない場合も表示対象から落とさない |
| popular | rank_targets と content_metrics をLEFT JOINし、content_metrics.click_count DESC, content_metrics.content_view_count DESC, id DESC | 人気順。クリック数と詳細閲覧数を優先する               |
| newest  | Beanは created_at DESC, id DESC。Articleは published_at DESC NULLS LAST, id DESC                                                 | 新着順                                |
| 未指定     | score と同じ                                                                                                                     | デフォルトはscore                        |

注意:

BeanRepository.SearchPublished と ArticleRepository.SearchPublished は、filter.Sort を必ず見る。

未知のsortはValidatorで弾くため、Repositoryでは原則受け取らない。

## ArticleRepository

| メソッド                | 引数                          | 戻り値                      | 用途                       |
|---------------------|-----------------------------|--------------------------|--------------------------|
| Create              | article *entity.Article     | error                    | AdminがArticleを作成する       |
| Update              | article *entity.Article     | error                    | AdminがArticleを更新する       |
| FindByID            | id uint64                   | *entity.Article, error   | Article詳細取得              |
| FindPublishedByID   | id uint64                   | *entity.Article, error   | 公開中Article詳細取得           |
| FindBySlug          | slug string                 | *entity.Article, error   | slugでArticleを取得する        |
| ListPublished       | limit int, offset int       | []*entity.Article, error | 公開中Article一覧取得           |
| SearchPublished     | filter ArticleSearchFilter  | []*entity.Article, error | Article検索                |
| FindByIDs           | ids []uint64                | []*entity.Article, error | ranking結果の実体取得           |
| ExistsByID          | id uint64                   | bool, error              | Admin・RankTarget作成時の存在確認 |
| ExistsPublishedByID | id uint64                   | bool, error              | 公開中Articleの存在確認          |
| UpdatePublished     | id uint64, isPublished bool | error                    | 公開状態を変更する                |

## GORM実装で使う取得条件

| メソッド                | GORM取得条件                                                                       | 何をするのか                                  |
|---------------------|--------------------------------------------------------------------------------|-----------------------------------------|
| FindByID            | WHERE id = ?                                                                   | Admin用に、公開状態に関係なくArticleを1件取得する         |
| FindPublishedByID   | WHERE id = ? AND is_published = true                                           | 一般ユーザー向けに、公開中Articleだけ取得する              |
| FindBySlug          | WHERE slug = ?                                                                 | URL上のslugからArticleを取得する                 |
| ListPublished       | WHERE is_published = true ORDER BY published_at DESC, id DESC LIMIT ? OFFSET ? | 公開中Articleを公開日時順に一覧取得する                 |
| SearchPublished     | WHERE is_published = true にfilter条件を追加                                         | 公開中Articleを検索条件で絞り込む                    |
| FindByIDs           | WHERE id IN ?                                                                  | ランキングやモーダル候補の結果から、複数Articleの実体を取得する     |
| ExistsByID          | WHERE id = ? の COUNT                                                           | Admin操作やRankTarget作成前にArticleが存在するか確認する |
| ExistsPublishedByID | WHERE id = ? AND is_published = true の COUNT                                   | 一般ユーザー操作前に、公開中Articleとして存在するか確認する       |
| UpdatePublished     | WHERE id = ?                                                                   | Articleの公開状態を変更する                       |

## usecaseからの呼び出し

| usecase                           | 呼び出し                               | 何をするのか                                                                    |
|-----------------------------------|------------------------------------|---------------------------------------------------------------------------|
| AdminArticleUsecase.CreateArticle | Create → AuditLogRepository.Create | Articleを作成する。作成時点ではRankTargetを作成しない。AuditLog作成に失敗してもArticle作成はrollbackしない |

| usecase                                           | 呼び出し                                                                                                                                                                                                          | 何をするのか                                                              |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>AdminArticleUsecase.UpdateArticle</code>    | <code>FindByID</code> → <code>Update</code> → <code>AuditLogRepository.Create</code>                                                                                                                          | 更新対象のArticleを確認してから更新する。AuditLog作成に失敗してもArticle更新はrollbackしない       |
| <code>AdminArticleUsecase.PublishArticle</code>   | <code>TxManager.WithinTx</code> 内で <code>UpdatePublished</code> → <code>RankTargetRepository.FindOrCreate</code> → <code>RankTargetRepository.UpdateActive</code> 。その後 <code>AuditLogRepository.Create</code> | Articleの公開状態とランキング対象の有効状態を同期する。AuditLog作成に失敗しても公開処理本体はrollbackしない   |
| <code>AdminArticleUsecase.UnpublishArticle</code> | <code>TxManager.WithinTx</code> 内で <code>UpdatePublished</code> → <code>RankTargetRepository.UpdateActive</code> 。その後 <code>AuditLogRepository.Create</code>                                                  | Articleの非公開状態とランキング対象の無効状態を同期する。AuditLog作成に失敗しても非公開処理本体はrollbackしない |
| <code>ArticleUsecase.GetArticleDetail</code>      | <code>FindPublishedByID</code> または <code>FindPublishedBySlug</code>                                                                                                                                           | 公開中Articleの詳細を取得する                                                  |
| <code>ArticleUsecase.ListArticles</code>          | <code>ListPublished</code>                                                                                                                                                                                    | 公開中Article一覧を取得する                                                   |
| <code>SearchUsecase.SearchArticles</code>         | <code>SearchPublished</code>                                                                                                                                                                                  | Article検索結果を返す。<br><code>ActionEventRepository</code> は直接呼ばない       |
| <code>RankingUsecase.ListArticleRanking</code>    | <code>ContentMetricRepository.ListRanking</code> → <code>RankTargetRepository</code> → <code>FindByIDs</code>                                                                                                 | ランキング指標からArticleのIDを取り出し、Article本体を取得する                             |
| <code>ModalUsecase.GetCandidates</code>           | <code>FindByIDs</code>                                                                                                                                                                                        | モーダル候補のRankTargetからArticle本体を取得する                                   |

| メソッド                             | 用途                 | 取得条件                                          |
|----------------------------------|--------------------|-----------------------------------------------|
| <code>FindPublishedBySlug</code> | ユーザー向けArticle詳細取得  | <code>slug = ? AND is_published = true</code> |
| <code>FindPublishedByID</code>   | ユーザー向け関連Article取得  | <code>id = ? AND is_published = true</code>   |
| <code>FindByID</code>            | 管理者向けArticle取得     | <code>id = ?</code>                           |
| <code>FindBySlug</code>          | 管理者向けslug重複確認・内部処理 | <code>slug = ?</code>                         |

```

type ArticleRepository interface {
    Create(ctx context.Context, article *entity.Article) error
    Update(ctx context.Context, article *entity.Article) error

    FindByID(ctx context.Context, id uint64) (*entity.Article, error)
    FindPublishedByID(ctx context.Context, id uint64) (*entity.Article, error)

    FindBySlug(ctx context.Context, slug string) (*entity.Article, error)
    FindPublishedBySlug(ctx context.Context, slug string) (*entity.Article, error)

    ListPublished(ctx context.Context, limit int, offset int) ([]*entity.Article, error)
    SearchPublished(ctx context.Context, filter ArticleSearchFilter) ([]*entity.Article, error)

    FindByIDs(ctx context.Context, ids []uint64) ([]*entity.Article, error)

    ExistsByID(ctx context.Context, id uint64) (bool, error)
    ExistsPublishedByID(ctx context.Context, id uint64) (bool, error)

    UpdatePublished(ctx context.Context, id uint64, isPublished bool) error
}

```

## BeanArticleRepository

| メソッド                | 引数                                        | 戻り値                | 用途                 |
|---------------------|-------------------------------------------|--------------------|--------------------|
| <code>Create</code> | <code>relation *entity.BeanArticle</code> | <code>error</code> | BeanとArticleを関連付ける |

|                 |                                    |                              |                       |
|-----------------|------------------------------------|------------------------------|-----------------------|
| Delete          | beanID uint64, articleID uint64    | error                        | 関連を削除する               |
| Exists          | beanID uint64, articleID uint64    | bool, error                  | 重複関連を確認する             |
| ListByBeanID    | beanID uint64, limit int           | []*entity.BeanArticle, error | Bean詳細に出す関連記事を取得する    |
| ListByArticleID | articleID uint64, limit int        | []*entity.BeanArticle, error | Article側から関連Beanを取得する |
| ReplaceByBeanID | beanID uint64, articleIDs []uint64 | error                        | Beanに紐づく関連記事を差し替える    |

## GORM実装で使う取得条件

| メソッド            | GORM取得条件                                                                                   | 何をするのか                           |
|-----------------|--------------------------------------------------------------------------------------------|----------------------------------|
| Exists          | WHERE bean_id = ? AND article_id = ?                                                       | 同じBeanとArticleの関連が既に存在しないか確認する   |
| ListByBeanID    | Preload("Article").Where("bean_id = ?", beanID).Order("display_order ASC").Limit(limit)    | Bean詳細に表示する関連記事を取得する             |
| ListByArticleID | Preload("Bean").Where("article_id = ?", articleID).Order("display_order ASC").Limit(limit) | Article側から関連Beanを取得する            |
| Delete          | WHERE bean_id = ? AND article_id = ?                                                       | 指定されたBeanとArticleの関連を削除する        |
| ReplaceByBeanID | transaction内で DELETE WHERE bean_id = ? → bulk insert                                       | 指定Beanに紐づく関連記事を一度削除し、新しい関連に差し替える |

## usecaseからの呼び出し

| usecase                                 | 呼び出し                                                                                                                            | 何をするのか                                                                                       |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| AdminRelationUsecase.CreateRelation     | BeanRepository.ExistsByID → ArticleRepository.ExistsByID → Exists → Create → AuditLogRepository.Create                          | BeanとArticleが存在すること、重複関連がないことを確認してから関連を1件作成する。1件作成ではTxを使わない。AuditLog作成に失敗しても関連作成はrollbackしない |
| AdminRelationUsecase.DeleteRelation     | Delete → AuditLogRepository.Create                                                                                              | BeanとArticleの関連を1件削除する。1件削除ではTxを使わない。AuditLog作成に失敗しても関連削除はrollbackしない                        |
| AdminRelationUsecase.ReplaceRelations   | BeanRepository.ExistsByID → 各 ArticleRepository.ExistsByID → TxManager.WithinTx内で ReplaceByBeanID。その後 AuditLogRepository.Create | Beanの関連記事をまとめて差し替える。既存関連削除と新規関連作成を伴うためTxで行う                                                  |
| AdminRelationUsecase.UpdateDisplayOrder | BeanRepository.ExistsByID → 各 ArticleRepository.ExistsByID → TxManager.WithinTx内で ReplaceByBeanID。その後 AuditLogRepository.Create | 関連Articleの表示順をまとめて更新する。全体差し替えなのでTxで行う                                                        |
| BeanUsecase.GetBeanDetail               | ListByBeanID                                                                                                                    | Bean詳細に表示する関連記事を取得する                                                                         |
| ArticleUsecase.GetArticleDetail         | ListByArticleID                                                                                                                 | Article詳細に表示する関連Beanを取得する                                                                    |

## RankTargetRepository

| メソッド             | 引数                                               | 戻り値                         | 用途                              |
|------------------|--------------------------------------------------|-----------------------------|---------------------------------|
| Create           | target *entity.RankTarget                        | error                       | RankTargetを作成する                 |
| FindByID         | id uint64                                        | *entity.RankTarget, error   | RankTargetをIDで取得する              |
| FindByContent    | contentType entity.ContentType, contentID uint64 | *entity.RankTarget, error   | content_type + content_id で取得する |
| FindOrCreate     | contentType entity.ContentType, contentID uint64 | *entity.RankTarget, error   | なければ作成する                        |
| FindByIDs        | ids []uint64                                     | []*entity.RankTarget, error | 複数RankTargetを取得する               |
| ListActiveByType | contentType entity.ContentType                   | []*entity.RankTarget, error | 有効な対象を種別別に取得する                  |

|                  |                          |             |                                          |
|------------------|--------------------------|-------------|------------------------------------------|
| ExistsActiveByID | id uint64                | bool, error | 保存・評価・イベント前に有効確認する ( WithTx またはtx対応実装方針) |
| UpdateActive     | id uint64, isActive bool | error       | ランキング対象の有効状態を切り替える                       |

## GORM実装で使う取得条件

| メソッド             | GORM取得条件                                                   | 何をするのか                                          |
|------------------|------------------------------------------------------------|-------------------------------------------------|
| FindByID         | WHERE id = ?                                               | RankTarget IDからランキング対象を取得する                     |
| FindByContent    | WHERE content_type = ? AND content_id = ?                  | Bean / Articleの実体IDからRankTargetを取得する            |
| FindOrCreate     | WHERE content_type = ? AND content_id = ? →<br>なければ Create | 指定されたBean / Articleに対応するRankTargetを取得し、なければ作成する |
| FindByIDs        | WHERE id IN ?                                              | 複数のRankTargetをまとめて取得する                          |
| ListActiveByType | WHERE content_type = ? AND is_active = true                | 指定種別の有効なランキング対象を取得する                            |
| ExistsActiveByID | WHERE id = ? AND is_active = true の COUNT                  | 保存・評価( WithTx またはtx対応実装方針)・イベント記録前に、有効な対象か確認する  |
| UpdateActive     | WHERE id = ?                                               | ランキング対象として有効かどうかを切り替える                          |

## usecaseからの呼び出し

| usecase                           | 呼び出し                                        | 何をするのか                       |
|-----------------------------------|---------------------------------------------|------------------------------|
| AdminBeanUsecase.CreateBean       | FindOrCreate(ContentTypeBean, beanID)       | 作成したBeanを共通ランキング対象として登録する    |
| AdminArticleUsecase.CreateArticle | FindOrCreate(ContentTypeArticle, articleID) | 作成したArticleを共通ランキング対象として登録する |
| EventUsecase.RecordEvent          | ExistsActiveByID                            | イベント記録前に、対象が有効なランキング対象か確認する  |
| SavedItemUsecase.Save             | ExistsActiveByID                            | 保存前に、保存対象が有効か確認する            |
| RatingUsecase.Rate                | ExistsActiveByID                            | 評価前に、評価対象が有効か確認する            |
| RankingBatchUsecase.Recalculate   | ListActiveByType                            | バッチ計算対象となる有効なRankTargetを取得する |
| ModalUsecase.GetCandidates        | FindByIDs                                   |                              |

## ActionEventRepository

| メソッド                     | 引数                                                                      | 戻り値                                | 用途                                                                                              |
|--------------------------|-------------------------------------------------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------|
| Create                   | event *entity.ActionEvent                                               | error                              | 行動イベントを1件記録する                                                                                   |
| BulkCreate               | events<br>[]*entity.ActionEvent                                         | error                              | 行動イベントをまとめて記録する                                                                                 |
| FindRecentByActor        | userID *uint64,<br>guestSessionID *uint64,<br>limit int                 | []*entity.ActionEvent,<br>error    | ユーザーまたはGuestの直近行動を取得する                                                                          |
| FindLastSearchHash       | userID *uint64,<br>guestSessionID *uint64                               | *string, error                     | 直近の re_search 条件hashを取得する。 re_search の記録判断は EventUsecase.RecordEvent で行う。 SearchUsecase では記録しない |
| AggregateContentMetrics  | periodStart time.Time,<br>periodEnd time.Time                           | []ContentMetricAggregate,<br>error | 直近期間のランキング指標を集計する<br>Repositoryは集計値を返すだけ。score<br>計算はUsecaseで行う                                 |
| AggregateUserInterest    | userID uint64, periodStart<br>time.Time, periodEnd<br>time.Time         | []InterestAggregate,<br>error      | Userの興味スコア材料を集計する                                                                               |
| AggregateGuestInterest   | guestSessionID uint64,<br>periodStart time.Time,<br>periodEnd time.Time | []InterestAggregate,<br>error      | Guestの興味スコア材料を集計する                                                                              |
| CountByActorAndTypeSince | userID *uint64,<br>guestSessionID *uint64,                              | int64, error                       | 特定actorのイベント数を数える                                                                               |

|                                        |                                                                                                               |                           |                |
|----------------------------------------|---------------------------------------------------------------------------------------------------------------|---------------------------|----------------|
|                                        | <code>eventType entity.EventType,</code><br><code>since time.Time</code>                                      |                           |                |
| <code>CountByTargetAndTypeSince</code> | <code>rankTargetID uint64,</code><br><code>eventType entity.EventType,</code><br><code>since time.Time</code> | <code>int64, error</code> | 特定対象のイベント数を数える |

## GORM実装で使う取得条件

| メソッド                                   | GORM取得条件                                                                                                                                                             | 何をしているのか                                                                                                                                                                                  |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Create</code>                    | <code>INSERT</code>                                                                                                                                                  | 行動イベントを1件保存する                                                                                                                                                                             |
| <code>BulkCreate</code>                | <code>CreateInBatches</code>                                                                                                                                         | 行動イベントをまとめて保存する                                                                                                                                                                           |
| <code>FindRecentByActor</code>         | <code>WHERE user_id = ? または</code><br><code>guest_session_id = ? ORDER BY</code><br><code>occurred_at DESC LIMIT ?</code>                                            | UserまたはGuestの直近行動を取得する                                                                                                                                                                    |
| <code>FindLastSearchHash</code>        | <code>WHERE event_type = 're_search'</code><br><code>AND actor条件 ORDER BY occurred_at</code><br><code>DESC LIMIT 1</code>                                            | 直近の検索条件hashを取得し、同じ検索条件の連続記録を防ぐ材料にする。re_searchの記録は <code>EventUsecase.RecordEvent</code> で行う。<br><code>SearchUsecase.SearchBeans / SearchArticles</code> では <code>re_search</code> を作成しない。 |
| <code>AggregateContentMetrics</code>   | <code>WHERE occurred_at &gt;= ? AND</code><br><code>occurred_at &lt; ? AND</code><br><code>rank_target_id IS NOT NULL GROUP</code><br><code>BY rank_target_id</code> | 指定期間の行動イベントを対象ごとに集計し、ランキング指標の材料を作る<br>Repositoryは集計値を返すだけ。score計算はUsecaseで行う                                                                                                              |
| <code>AggregateUserInterest</code>     | <code>WHERE user_id = ? AND</code><br><code>occurred_at &gt;= ? AND occurred_at</code><br><code>&lt; ?</code>                                                        | 指定Userの行動から興味スコアの材料を集計する                                                                                                                                                                  |
| <code>AggregateGuestInterest</code>    | <code>WHERE guest_session_id = ? AND</code><br><code>occurred_at &gt;= ? AND occurred_at</code><br><code>&lt; ?</code>                                               | 指定Guestの行動から興味スコアの材料を集計する                                                                                                                                                                 |
| <code>CountByActorAndTypeSince</code>  | <code>WHERE actor条件 AND event_type =</code><br><code>? AND occurred_at &gt;= ?</code>                                                                                | 指定actorの特定イベント回数を数える                                                                                                                                                                      |
| <code>CountByTargetAndTypeSince</code> | <code>WHERE rank_target_id = ? AND</code><br><code>event_type = ? AND occurred_at &gt;=</code><br><code>?</code>                                                     | 指定対象の特定イベント回数を数える                                                                                                                                                                         |

## AggregateContentMetricsで集計する項目

Repositoryは集計値を返すだけ。

score計算はUsecaseで行う。

re\_search を RankTarget別metricsに含めない。

| 集計項目                 | 条件                                                       |
|----------------------|----------------------------------------------------------|
| ImpressionCount      | <code>event_type = 'impression'</code>                   |
| ContentViewCount     | <code>event_type = 'content_view'</code>                 |
| ClickCount           | <code>event_type = 'click'</code>                        |
| StayTotalMs          | <code>event_type = 'stay' の SUM(dwel_ms)</code>          |
| SaveCount            | <code>event_type = 'save'</code>                         |
| RatingCount          | <code>event_type = 'rating'</code>                       |
| GoodCount            | <code>event_type = 'rating' AND rating_score = 1</code>  |
| BadCount             | <code>event_type = 'rating' AND rating_score = -1</code> |
| ModalImpressionCount | <code>event_type = 'modal_impression'</code>             |
| ModalClickCount      | <code>event_type = 'modal_click'</code>                  |
| ModalCloseCount      | <code>event_type = 'modal_close'</code>                  |

re\_search は検索条件変更イベントであり、RankTargetIDを持たない。

そのため、AggregateContentMetricsでは集計しない。

## usecaseからの呼び出し

|         |      |          |
|---------|------|----------|
| usecase | 呼び出し | 何をしているのか |
|---------|------|----------|

|                                                       |                                                                                                                             |                                                                               |
|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>EventUsecase.RecordEvent</code>                 | <code>RankTargetRepository.ExistsActiveByID</code> → <code>EventDedupRepository.SetIfNotExists</code> → <code>Create</code> | 対象が有効か確認し、重複でなければ閲覧イベントを保存する                                                  |
| <code>EventUsecase.RecordImpression</code>            | <code>EventDedupRepository.SetIfNotExists</code> → <code>Create</code>                                                      | 同じ表示イベントの重複を防ぎ、初回だけimpressionを保存する                                            |
| <code>EventUsecase.RecordStay</code>                  | 滞在秒数検証 → <code>Create</code>                                                                                                | 滞在時間が有効範囲なら dwell/stay イベントを保存する                                              |
| <code>SearchUsecase.ReSearch(re_search)</code>        | <code>ActionEventRepository.FindLastSearchHash</code> → <code>ActionEventRepository.Create</code>                           | POST /events で受け取った再検索イベントを記録する。直前の検索条件hashと比較し、条件が変わった場合のみ re_search として保存する |
| <code>SearchUsecase.SearchBeans/SearchArticles</code> | 呼び出さない                                                                                                                      | 検索APIは検索結果を返すだけ。<br><code>re_search</code> 記録は <code>POST /events</code> に寄せる |
| <code>RankingBatchUsecase.Recalculate</code>          | <code>AggregateContentMetrics</code>                                                                                        | 行動イベントを集計し、ランキング指標の材料にする<br>Repositoryは集計値を返すだけ。<br>score計算はUsecaseで行う        |
| <code>InterestBatchUsecase.RecalculateUser</code>     | <code>AggregateUserInterest</code>                                                                                          | Userの行動を集計し、興味スコアを再計算する                                                       |
| <code>InterestBatchUsecase.RecalculateGuest</code>    | <code>AggregateGuestInterest</code>                                                                                         |                                                                               |

## ModalDisplayLogRepository

ModalDisplayLogRepository は、モーダル表示ログを作成し、modal\_click / modal\_close 時に本人または本人 GuestSessionの表示ログだけを更新する。

modal\_display\_log\_id だけで取得・更新してはいけない。

必ず user\_id または guest\_session\_id による actor条件を付ける。

| メソッド                                 | 引数                                                                                                             | 戻り値                                         | 用途                                                           |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------|--------------------------------------------------------------|
| <code>Create</code>                  | <code>ctx context.Context, log *entity.ModalDisplayLog</code>                                                  | <code>error</code>                          | モーダル表示ログを作成する                                                |
| <code>FindByIDForActor</code>        | <code>ctx context.Context, id uint64, userID *uint64, guestSessionID *uint64</code>                            | <code>*entity.ModalDisplayLog, error</code> | modal_click / modal_close 時に、本人または本人 GuestSessionの表示ログだけ取得する |
| <code>MarkClickedForActor</code>     | <code>ctx context.Context, id uint64, userID *uint64, guestSessionID *uint64, clickedAt time.Time</code>       | <code>error</code>                          | 本人または本人 GuestSessionの表示ログだけ clicked_at を更新する                 |
| <code>MarkClosedForActor</code>      | <code>ctx context.Context, id uint64, userID *uint64, guestSessionID *uint64, closedAt time.Time</code>        | <code>error</code>                          | 本人または本人 GuestSessionの表示ログだけ closed_at を更新する                  |
| <code>CountShownInSession</code>     | <code>ctx context.Context, userID *uint64, guestSessionID *uint64, since time.Time</code>                      | <code>int64, error</code>                   | 同一Actorの一定期間内表示回数を数える                                        |
| <code>CountShownOnPage</code>        | <code>ctx context.Context, userID *uint64, guestSessionID *uint64, pagePath string, since time.Time</code>     | <code>int64, error</code>                   | 同一Actor・同一ページの一定期間内表示回数を数える                                  |
| <code>WasTargetShownRecently</code>  | <code>ctx context.Context, userID *uint64, guestSessionID *uint64, rankTargetID uint64, since time.Time</code> | <code>bool, error</code>                    | 同一候補を直近表示済みか確認する                                             |
| <code>WasTargetClosedRecently</code> | <code>ctx context.Context, userID *uint64, guestSessionID *uint64, rankTargetID uint64, since time.Time</code> | <code>bool, error</code>                    | 同一候補が直近で閉じられていないか確認する                                        |

## GORM実装で使う取得条件

| メソッド                             | GORM取得条件                 | 何をしているのか                                          |
|----------------------------------|--------------------------|---------------------------------------------------|
| <code>FindByIDForActor</code>    | WHERE id = ? AND actor条件 | 他人の modal_display_log_id を指定できないように、本人の表示ログだけ取得する |
| <code>MarkClickedForActor</code> | WHERE id = ? AND actor条件 | 本人の表示ログだけ clicked_at を更新する                        |

| メソッド                                 | GORM取得条件                                                | 何をしているのか                  |
|--------------------------------------|---------------------------------------------------------|---------------------------|
| <code>MarkClosedForActor</code>      | WHERE id = ? AND actor条件                                | 本人の表示ログだけ closed_at を更新する |
| <code>CountShownInSession</code>     | WHERE actor条件 AND shown_at >= ?                         | 同一Actorの表示回数を数える          |
| <code>CountShownOnPage</code>        | WHERE actor条件 AND page_path = ? AND shown_at >= ?       | 同一ページの表示回数を数える            |
| <code>WasTargetShownRecently</code>  | WHERE actor条件 AND rank_target_id = ? AND shown_at >= ?  | 同一候補を直近表示済みか確認する          |
| <code>WasTargetClosedRecently</code> | WHERE actor条件 AND rank_target_id = ? AND closed_at >= ? | 同一候補が直近で閉じられていないか確認する     |

## actor条件

| Actor              | 条件                                                    |
|--------------------|-------------------------------------------------------|
| <code>User</code>  | <code>user_id = ? AND guest_session_id IS NULL</code> |
| <code>Guest</code> | <code>user_id IS NULL AND guest_session_id = ?</code> |

## usecaseからの呼び出し

| usecase                                       | 呼び出し                                                                                                                    | 何をしているのか                                                                                                                     |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>ModalUsecase.ShowModal</code>           | ModalDisplayLogRepository.Create → ActionEventRepository.Create(modal_impression) → ModalSuppressionRepository.SetShown | モーダル表示ログを作成する。modal_impression event作成に失敗しても表示ログはrollbackしない。Redis抑制保存はDB Tx外で行う                                             |
| <code>ModalUsecase.ClickModal</code>          | MarkClickedForActor → ActionEventRepository.Create(modal_click)                                                         | actor条件付きUPDATEで本人または本人GuestSessionの表示ログだけ clicked_atを更新する。modal_click event作成に失敗してもclicked_atはrollbackしない                   |
| <code>ModalUsecase.CloseModal</code>          | MarkClosedForActor → ActionEventRepository.Create(modal_close) → ModalSuppressionRepository.SetClosed                   | actor条件付きUPDATEで本人または本人GuestSessionの表示ログだけ closed_atを更新する。modal_close event作成に失敗してもclosed_atはrollbackしない。Redis抑制保存はDB Tx外で行う |
| <code>ModalUsecase.CheckRecentlyShown</code>  | WasTargetShownRecently                                                                                                  | 同じ候補が直近表示済みか確認する                                                                                                             |
| <code>ModalUsecase.CheckRecentlyClosed</code> | WasTargetClosedRecently                                                                                                 | 同じ候補が直近で閉じられていないか確認する                                                                                                        |

## ModalBlockLogRepository

| メソッド                            | 引数                                                             | 戻り値                                         | 用途                                          |
|---------------------------------|----------------------------------------------------------------|---------------------------------------------|---------------------------------------------|
| <code>Create</code>             | <code>log *entity.ModalBlockLog, ModalUsecase.CanShow</code>   | <code>error</code>                          | 表示上限、保存済み、直近表示済み、閉じたばかりなどでモーダルを表示しないと判断したとき |
| <code>ListByActor</code>        | <code>userID *uint64, guestSessionID *uint64, limit int</code> | <code>[]*entity.ModalBlockLog, error</code> | actor別の非表示履歴を取得する                           |
| <code>CountByReasonSince</code> | <code>reason entity.ModalBlockReason, since time.Time</code>   | <code>int64, error</code>                   | 非表示理由ごとの件数を数える                              |
| <code>ListByCandidate</code>    | <code>rankTargetID uint64, limit int</code>                    | <code>[]*entity.ModalBlockLog, error</code> | 候補別の非表示履歴を取得する                              |

## GORM実装で使う取得条件

| メソッド                     | GORM取得条件                                                                                 | 何をしているのか                              |
|--------------------------|------------------------------------------------------------------------------------------|---------------------------------------|
| <code>Create</code>      | <code>INSERT</code>                                                                      | モーダルを表示しなかった理由を1件保存する                 |
| <code>ListByActor</code> | <code>WHERE user_id = ? または guest_session_id = ? ORDER BY blocked_at DESC LIMIT ?</code> | 指定UserまたはGuestSessionの非表示履歴を新しい順に取得する |



|                    |                                                                     |                            |
|--------------------|---------------------------------------------------------------------|----------------------------|
| CountByReasonSince | WHERE reason = ? AND blocked_at >= ?                                | 指定期間内に、特定の非表示理由が何回発生したか数える |
| ListByCandidate    | WHERE candidate_rank_target_id = ? ORDER BY blocked_at DESC LIMIT ? | 特定候補が非表示になった履歴を取得する        |

## usecaseからの呼び出し

| usecase                          | 呼び出し                                 | 何をしているのか                         |
|----------------------------------|--------------------------------------|----------------------------------|
| ModalUsecase.CanShow             | 表示不可の理由がある場合 Create                  | モーダルを表示しなかった理由を記録し、後から分析できるようにする |
| AdminAnalysisUsecase.ModalBlocks | CountByReasonSince / ListByCandidate | 非表示理由ごとの件数や候補別の非表示履歴を分析する        |

## SavedItemRepository

| メソッド                | 引数                                                      | 戻り値                        | 用途           |
|---------------------|---------------------------------------------------------|----------------------------|--------------|
| Create              | item *entity.SavedItem                                  | error                      | 初回保存を作成する    |
| FindByUserAndTarget | userID uint64, rankTargetID uint64                      | *entity.SavedItem, error   | 保存状態を取得する    |
| SaveOrRestore       | userID uint64, rankTargetID uint64, now time.Time       | *entity.SavedItem, error   | 新規保存または再保存する |
| Remove              | userID uint64, rankTargetID uint64, removedAt time.Time | error                      | 保存解除する       |
| ListActiveByUserID  | userID uint64, limit int, offset int                    | []*entity.SavedItem, error | 保存中一覧を取得する   |
| ExistsActive        | userID uint64, rankTargetID uint64                      | bool, error                | 保存済みか確認する    |
| CountActiveByTarget | rankTargetID uint64                                     | int64, error               | 対象ごとの保存数を数える |

## GORM実装で使う取得条件

| メソッド                | GORM取得条件                                                                                   | 何をしているのか                       |
|---------------------|--------------------------------------------------------------------------------------------|--------------------------------|
| FindByUserAndTarget | WHERE user_id = ? AND rank_target_id = ?                                                   | 指定Userが対象を保存した履歴を取得する          |
| SaveOrRestore       | WHERE user_id = ? AND rank_target_id = ? → なければCreate、removed_atありならNULLへ更新                | 未保存なら新規保存し、過去に保存解除済みなら再保存状態に戻す |
| Remove              | WHERE user_id = ? AND rank_target_id = ? AND removed_at IS NULL                            | 現在保存中の対象だけを保存解除する              |
| ListActiveByUserID  | Preload("RankTarget").Where("user_id = ? AND removed_at IS NULL").Order("updated_at DESC") | 指定Userの保存中アイテムを新しい順に取得する       |
| ExistsActive        | WHERE user_id = ? AND rank_target_id = ? AND removed_at IS NULL                            | 指定Userが対象を現在保存中か確認する           |
| CountActiveByTarget | WHERE rank_target_id = ? AND removed_at IS NULL                                            | 指定対象が現在何件保存されているか数える           |

## usecaseからの呼び出し

| usecase                   | 呼び出し                                                                                       | 何をしているのか                                                                     |
|---------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| SavedItemUsecase.Save     | RankTargetRepository.ExistsActiveByID → SaveOrRestore → ActionEventRepository.Create(save) | 保存対象が有効か確認し、保存または再保存する。save event作成は保存成功後に試みるが、event作成に失敗しても保存本体はrollbackしない |
| SavedItemUsecase.Remove   | Remove                                                                                     | 指定Userの保存中アイテムを保存解除する。保存解除は単体更新で完結し、save eventは作成しない                         |
| SavedItemUsecase.List     | ListActiveByUserID                                                                         | 指定Userの保存一覧を取得する                                                             |
| ModalUsecase.ExcludeSaved | ExistsActive                                                                               | モーダル候補から保存済み対象を除外する                                                          |



## RatingRepository

| メソッド                | 引数                                                                          | 戻り値                    | 用途              |
|---------------------|-----------------------------------------------------------------------------|------------------------|-----------------|
| Create              | rating *entity.Rating                                                       | error                  | 初回評価を作成する       |
| FindByUserAndTarget | userID uint64, rankTargetID uint64                                          | *entity.Rating, error  | 自分の評価を取得する      |
| Upsert              | userID uint64, rankTargetID uint64, score entity.RatingScore, now time.Time | *entity.Rating, error  | 初回評価または再評価する    |
| Delete              | userID uint64, rankTargetID uint64                                          | error                  | 評価取消を行う場合に使う    |
| CountByTarget       | rankTargetID uint64                                                         | int64, error           | 評価数を数える         |
| AggregateByTarget   | rankTargetID uint64                                                         | RatingAggregate, error | Good / Badを集計する |

## GORM実装で使う取得条件

| メソッド                | GORM取得条件                                                               | 何をしているのか                     |
|---------------------|------------------------------------------------------------------------|------------------------------|
| FindByUserAndTarget | WHERE user_id = ? AND rank_target_id = ?                               | 指定Userが対象に付けた評価を取得する         |
| Upsert              | WHERE user_id = ? AND rank_target_id = ? → なければ Create、あればscore更新      | 初回評価なら作成し、既に評価済みならscoreを更新する |
| Delete              | WHERE user_id = ? AND rank_target_id = ?                               | 指定Userの対象に対する評価を削除する         |
| CountByTarget       | WHERE rank_target_id = ?                                               | 指定対象に付いた評価数を数える              |
| AggregateByTarget   | WHERE rank_target_id = ? で COUNT , SUM(score = 1) , SUM(score = -1) 相当 | 指定対象のGood数、Bad数、評価数を集計する     |

## usecaseからの呼び出し

| usecase                         | 呼び出し                                                                                  | 何をしているのか                                                                                  |
|---------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| RatingUsecase.Rate              | RankTargetRepository.ExistsActiveByID → Upsert → ActionEventRepository.Create(rating) | 評価対象が有効か確認し、Good/Bad評価を作成または更新する。rating event 作成は評価成功後に試みるが、event作成に失敗しても評価本体はrollbackしない |
| RatingUsecase.GetMyRating       | FindByUserAndTarget                                                                   | 自分が対象に付けた評価を取得する                                                                          |
| RatingUsecase.DeleteRating      | Delete                                                                                | 自分が対象に付けた評価を削除する。Txは使わない                                                                  |
| RankingBatchUsecase.Recalculate | 原則 ActionEventRepository.AggregateContentMetrics を使うため直接使わなくてもよい                      | 評価集計はActionEventまたはratings集計方針に合わせて行う                                                     |

| メソッド                | 引数                                                     | 戻り値                            | 用途                   |
|---------------------|--------------------------------------------------------|--------------------------------|----------------------|
| Upsert              | metric *entity.ContentMetric                           | error                          | 指標を作成または更新する         |
| BulkUpsert          | metrics []*entity.ContentMetric                        | error                          | バッチで指標を一括更新する        |
| FindByRankTargetID  | rankTargetID uint64                                    | *entity.ContentMetric, error   | 対象の指標を取得する           |
| FindByRankTargetIDs | rankTargetIDs []uint64                                 | []*entity.ContentMetric, error | 複数対象の指標を取得する         |
| ListRanking         | contentType *entity.ContentType, limit int, offset int | []*entity.ContentMetric, error | 最新30日集計のランキング一覧を取得する |
| ListTopByScore      | limit int                                              | []*entity.ContentMetric, error | TOPランキング取得           |
| LatestCalculatedAt  | なし                                                     | *time.Time, error              | 最新集計日時を取得する          |

## GORM実装で使う取得条件

| メソッド | GORM取得条件 | 何をしているのか |
|------|----------|----------|
|------|----------|----------|

|                     |                                                                                |                                  |
|---------------------|--------------------------------------------------------------------------------|----------------------------------|
| Upsert              | ON CONFLICT(rank_target_id) DO UPDATE                                          | 1つのRankTargetにつき最新の指標を作成または更新する  |
| BulkUpsert          | batchで ON CONFLICT(rank_target_id) DO UPDATE                                   | バッチで計算した複数の指標をまとめて作成または更新する      |
| FindByRankTargetID  | WHERE rank_target_id = ?                                                       | 指定RankTargetの現在の指標を取得する          |
| FindByRankTargetIDs | WHERE rank_target_id IN ?                                                      | 複数RankTargetの指標をまとめて取得する         |
| ListRanking         | Preload("RankTarget").JOIN rank_targets して content_type 条件、ORDER BY score DESC | BeanまたはArticleのランキング一覧をスコア順に取得する |
| ListTopByScore      | ORDER BY score DESC LIMIT ?                                                    | 種別を問わず、スコア上位の対象を取得する             |
| LatestCalculatedAt  | ORDER BY calculated_at DESC LIMIT 1                                            | 最後にランキング指標を計算した日時を取得する           |

## usecaseからの呼び出し

| usecase                         | 呼び出し                                                                  | 何をしているのか                          |
|---------------------------------|-----------------------------------------------------------------------|-----------------------------------|
| RankingUsecase.ListRanking      | ListRanking                                                           | ランキング画面に表示する指標一覧を取得する             |
| RankingUsecase.GetTargetMetric  | FindByRankTargetID                                                    | 指定対象のランキング指標を取得する                 |
| RankingBatchUsecase.Recalculate | ActionEventRepository.AggregateContentMetrics → score 計算 → BulkUpsert | 行動イベントを集計し、ランキングスコアを計算し、指標を一括保存する |
| ModalUsecase.GetCandidates      | FindByRankTargetIDs / ListTopByScore                                  |                                   |

## ContentMetricRepository

| メソッド                | 引数                                                     | 戻り値                            | 用途                                      |
|---------------------|--------------------------------------------------------|--------------------------------|-----------------------------------------|
| Upsert              | metric *entity.ContentMetric                           | error                          | 1つのRankTargetに対するランキング指標を作成または更新する      |
| BulkUpsert          | metrics []*entity.ContentMetric                        | error                          | RankingBatchで集計した複数のランキング指標を一括作成または更新する |
| FindByRankTargetID  | rankTargetID uint64                                    | *entity.ContentMetric, error   | 指定RankTargetのランキング指標を取得する               |
| FindByRankTargetIDs | rankTargetIDs []uint64                                 | []*entity.ContentMetric, error | 複数RankTargetのランキング指標をまとめて取得する           |
| ListRanking         | contentType *entity.ContentType, limit int, offset int | []*entity.ContentMetric, error | Bean / Article / 総合ランキングをscore順で取得する    |
| ListTopByScore      | limit int                                              | []*entity.ContentMetric, error | TOP表示やモーダル候補用にscore上位のランキング指標を取得する      |
| LatestCalculatedAt  | なし                                                     | *time.Time, error              | 最後にランキング指標を計算した日時を取得する                  |

## GORM実装で使う取得条件

| メソッド               | GORM取得条件                                     | 何をするのか                          |
|--------------------|----------------------------------------------|---------------------------------|
| Upsert             | ON CONFLICT(rank_target_id) DO UPDATE        | 1つのRankTargetにつき最新の指標を作成または更新する |
| BulkUpsert         | batchで ON CONFLICT(rank_target_id) DO UPDATE | バッチで計算した複数の指標をまとめて作成または更新する     |
| FindByRankTargetID | WHERE rank_target_id = ?                     | 指定RankTargetの現在の指標を取得する         |

| メソッド                | GORM取得条件                                                                                                                                                                           | 何をするのか                           |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| FindByRankTargetIDs | WHERE rank_target_id IN ?                                                                                                                                                          | 複数RankTargetの指標をまとめて取得する         |
| ListRanking         | JOIN rank_targets ON rank_targets.id = content_metrics.rank_target_id。<br>content_type条件がある場合はWHEREで絞り、ORDER BY content_metrics.score<br>DESC, content_metrics.rank_target_id DESC | BeanまたはArticleのランキング一覧をスコア順に取得する |
| ListTopByScore      | ORDER BY score DESC, rank_target_id DESC LIMIT ?                                                                                                                                   | 種別を問わず、スコア上位の対象を取得する             |
| LatestCalculatedAt  | ORDER BY calculated_at DESC LIMIT 1                                                                                                                                                | 最後にランキング指標を計算した日時を取得する           |

## usecaseからの呼び出し

| usecase                                   | 呼び出し                                                                                                                                      | 何をしているのか                                                               |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| RankingBatchUsecase.Recalculate           | TxManager.WithinTx内で<br>ContentMetricRepository.BulkUpsert →<br>BatchRunRepository.MarkSuccess                                            | 行動ログから計算したランキング指標を保存し、BatchRunを成功状態に更新する。集計結果と成功状態を一致させるため、この2つだけTxで行う |
| RankingUsecase.ListBeanRanking            | ContentMetricRepository.ListRanking →<br>RankTargetRepository.FindByIDs →<br>BeanRepository.FindByIDs                                     | Beanランキングを表示する                                                         |
| RankingUsecase.ListArticleRanking         | ContentMetricRepository.ListRanking →<br>RankTargetRepository.FindByIDs →<br>ArticleRepository.FindByIDs                                  | Articleランキングを表示する                                                      |
| RankingUsecase.ListTopRanking             | ContentMetricRepository.ListTopByScore →<br>RankTargetRepository.FindByIDs →<br>BeanRepository.FindByIDs /<br>ArticleRepository.FindByIDs | TOPランキングを表示する                                                          |
| ModalUsecase.GetCandidates                | ContentMetricRepository.ListTopByScore または<br>ListRanking                                                                                 | 推薦モーダル候補をスコア順に取得する                                                     |
| RecommendationUsecase.ListRecommendations | ContentMetricRepository.ListRanking または<br>FindByRankTargetIDs                                                                            | 推薦一覧のスコア情報を取得する                                                        |

## InterestProfileRepository

| メソッド               | 引数                                 | 戻り値                              | 用途               |
|--------------------|------------------------------------|----------------------------------|------------------|
| Upsert             | profile *entity.InterestProfile    | error                            | 興味スコアを作成または更新する  |
| BulkUpsert         | profiles []*entity.InterestProfile | error                            | 興味スコアを一括更新する     |
| FindByUser         | userID uint64                      | []*entity.InterestProfile, error | Userの興味スコアを取得する  |
| FindByGuestSession | guestSessionID uint64              | []*entity.InterestProfile, error | Guestの興味スコアを取得する |
| ListTopByUser      | userID uint64, limit int           | []*entity.InterestProfile, error | Userの上位興味軸を取得する  |
| ListTopByGuest     | guestSessionID uint64, limit int   | []*entity.InterestProfile, error | Guestの上位興味軸を取得する |
| DeleteExpired      | now time.Time                      | int64, error                     | 期限切れGuest興味を削除する |

## GORM実装で使う取得条件

| メソッド       | GORM取得条件                                                                                   | 何をしているのか                         |
|------------|--------------------------------------------------------------------------------------------|----------------------------------|
| Upsert     | Userなら (user_id, dimension, value)、Guestなら<br>(guest_session_id, dimension, value) でUpsert | UserまたはGuestの興味軸ごとのスコアを作成または更新する |
| BulkUpsert | batch upsert                                                                               | バッチで計算した興味スコアをまとめて保存する           |
| FindByUser | WHERE user_id = ? ORDER BY score DESC                                                      | 指定Userの興味スコアを高い順に取得する            |

|                    |                                                        |                               |
|--------------------|--------------------------------------------------------|-------------------------------|
| FindByGuestSession | WHERE guest_session_id = ? ORDER BY score DESC         | 指定GuestSessionの興味スコアを高い順に取得する |
| ListTopByUser      | WHERE user_id = ? ORDER BY score DESC LIMIT ?          | 指定Userの上位興味軸だけを取得する           |
| ListTopByGuest     | WHERE guest_session_id = ? ORDER BY score DESC LIMIT ? | 指定GuestSessionの上位興味軸だけを取得する   |
| DeleteExpired      | WHERE expires_at IS NOT NULL AND expires_at < ?        | 有効期限切れのGuest興味スコアを削除する        |

## usecaseからの呼び出し

| usecase                                           | 呼び出し                                                                          | 何をしているのか                                                                  |
|---------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| InterestBatchUsecase.Recalculate                  | BulkUpsert →<br>BatchRunRepository.MarkSuccess →<br>AuditLogRepository.Create | User/Guestごとの興味プロフィールを再計算して保存する。<br>InterestProfileは再計算可能な派生データなのでTxは使わない |
| RecommendationUsecase.ListRecommendations         | ListTopByUser または ListTopByGuest                                              | 推薦一覧のためにUser/Guestの興味スコア上位を取得する                                           |
| ModalUsecase.GetCandidates                        | ListTopByUser または ListTopByGuest                                              | モーダル候補選定のために興味スコアを取得する                                                    |
| CleanupUsecase.DeleteExpiredGuestInterestProfiles | DeleteExpiredGuestProfiles                                                    | 期限切れGuestSessionに紐づく興味プロフィールを削除する                                         |

## BatchRunRepository

| メソッド                 | 引数                                                                        | 戻り値                          | 用途                 |
|----------------------|---------------------------------------------------------------------------|------------------------------|--------------------|
| Create               | run *entity.BatchRun                                                      | error                        | バッチ開始ログを作成する       |
| MarkSuccess          | id uint64, finishedAt time.Time, rowsProcessed int64                      | error                        | バッチ成功として更新する       |
| MarkFailed           | id uint64, finishedAt time.Time, rowsProcessed int64, errorMessage string | error                        | バッチ失敗として更新する       |
| FindLatestByJobName  | jobName string                                                            | *entity.BatchRun,<br>error   | 最新実行履歴を取得する        |
| FindRunningByJobName | jobName string                                                            | *entity.BatchRun,<br>error   | 実行中バッチを取得する        |
| List                 | limit int, offset int                                                     | []*entity.BatchRun,<br>error | Admin画面でバッチ履歴を取得する |

## GORM実装で使う取得条件

| メソッド                 | GORM取得条件                                                                   | 何をしているのか                  |
|----------------------|----------------------------------------------------------------------------|---------------------------|
| FindLatestByJobName  | WHERE job_name = ? ORDER BY started_at DESC LIMIT 1                        | 指定jobの最新実行履歴を取得する         |
| FindRunningByJobName | WHERE job_name = ? AND status = 'running' ORDER BY started_at DESC LIMIT 1 | 指定jobが現在実行中か確認する          |
| MarkSuccess          | WHERE id = ?                                                               | 指定バッチ実行ログを成功状態に更新する       |
| MarkFailed           | WHERE id = ?                                                               | 指定バッチ実行ログを失敗状態に更新する       |
| List                 | ORDER BY started_at DESC LIMIT ? OFFSET ?                                  | Admin画面でバッチ実行履歴を新しい順に取得する |

## usecaseからの呼び出し

AdminBatchUsecaseが RankingBatchUsecase と InterestBatchUsecase を順番に呼ぶ

| usecase                           | 呼び出し                                      | 何をしているのか                         |
|-----------------------------------|-------------------------------------------|----------------------------------|
| RankingBatchUsecase.Recalculate   | Create → 成功時 MarkSuccess / 失敗時 MarkFailed | バッチ開始時に履歴を作成し、終了時に成功または失敗として更新する |
| AdminBatchUsecase.RunRankingBatch | FindRunningByJobName → Create             | 手動実行前に同じjobが実行中でないか確認し、実行履歴を作成する |

|                                              |                                  |                      |
|----------------------------------------------|----------------------------------|----------------------|
| <code>AdminBatchUsecase.ListBatchRuns</code> | <code>List</code>                | Admin画面でバッチ実行履歴を取得する |
| <code>AdminBatchUsecase.GetLatestRun</code>  | <code>FindLatestByJobName</code> | 指定jobの最新実行結果を取得する    |

## AuditLogRepository

| メソッド                         | 引数                                                                           | 戻り値                                    | 用途              |
|------------------------------|------------------------------------------------------------------------------|----------------------------------------|-----------------|
| <code>Create</code>          | <code>log *entity.AuditLog</code>                                            | <code>error</code>                     | 監査ログを作成する       |
| <code>FindByID</code>        | <code>id uint64</code>                                                       | <code>*entity.AuditLog, error</code>   | 監査ログ詳細を取得する     |
| <code>List</code>            | <code>filter AuditLogFilter</code>                                           | <code>[]*entity.AuditLog, error</code> | 監査ログ一覧を検索する     |
| <code>ListByActor</code>     | <code>actorType entity.AuditActorType, actorUserID *uint64, limit int</code> | <code>[]*entity.AuditLog, error</code> | 操作者別に取得する       |
| <code>ListByTarget</code>    | <code>targetType string, targetID uint64, limit int</code>                   | <code>[]*entity.AuditLog, error</code> | 操作対象別に取得する      |
| <code>ListByRequestID</code> | <code>requestID string</code>                                                | <code>[]*entity.AuditLog, error</code> | request_idで追跡する |

## GORM実装で使う取得条件

| メソッド                         | GORM取得条件                                                                                                                                                                  | 何をしているのか                      |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| <code>FindByID</code>        | <code>WHERE id = ?</code>                                                                                                                                                 | 監査ログIDから詳細を取得する               |
| <code>List</code>            | filterに応じて <code>actor_type</code> , <code>actor_user_id</code> , <code>action</code> , <code>target_type</code> , <code>target_id</code> , <code>created_at</code> 条件を付与 | 条件に一致する監査ログ一覧を検索する            |
| <code>ListByActor</code>     | <code>WHERE actor_type = ? AND actor_user_id = ? ORDER BY created_at DESC LIMIT ?</code>                                                                                  | 指定した操作者の監査ログを新しい順に取得する        |
| <code>ListByTarget</code>    | <code>WHERE target_type = ? AND target_id = ? ORDER BY created_at DESC LIMIT ?</code>                                                                                     | 指定した操作対象に関する監査ログを新しい順に取得する    |
| <code>ListByRequestID</code> | <code>WHERE request_id = ? ORDER BY created_at ASC</code>                                                                                                                 | 同じrequest_idに紐づく監査ログを時系列で取得する |

## usecaseからの呼び出し

| usecase                                        | 呼び出し                                        | 何をしているのか                                                                     |
|------------------------------------------------|---------------------------------------------|------------------------------------------------------------------------------|
| <code>AuthUsecase.Login</code>                 | <code>Create(login)</code>                  | ログイン操作を監査ログに記録する。AuditLog作成に失敗してもLogin本体はrollbackしない                         |
| <code>AuthUsecase.LogoutCurrentFamily</code>   | <code>Create(logout)</code>                 | ログアウト操作を監査ログに記録する。AuditLog作成に失敗してもlogout本体はrollbackしない                       |
| <code>AuthUsecase.LogoutAllDevices</code>      | <code>Create(logout_all_devices)</code>     | 全端末ログアウト操作を監査ログに記録する。AuditLog作成に失敗してもlogout本体はrollbackしない                    |
| <code>AuthUsecase.RefreshReuseDetected</code>  | <code>Create(refresh_reuse_detected)</code> | RefreshTokenのreuse検知をセキュリティイベントとして記録する。AuditLog作成に失敗してもreuse対応本体はrollbackしない |
| <code>AdminBeanUsecase.CreateBean</code>       | <code>Create(bean_create)</code>            | AdminによるBean作成操作を記録する。AuditLog作成に失敗してもBean作成はrollbackしない                     |
| <code>AdminBeanUsecase.UpdateBean</code>       | <code>Create(bean_update)</code>            | AdminによるBean更新操作を記録する。AuditLog作成に失敗してもBean更新はrollbackしない                     |
| <code>AdminBeanUsecase.PublishBean</code>      | <code>Create(bean_publish)</code>           | AdminによるBean公開操作を記録する。AuditLog作成に失敗してもBean公開処理はrollbackしない                   |
| <code>AdminBeanUsecase.UnpublishBean</code>    | <code>Create(bean_unpublish)</code>         | AdminによるBean非公開操作を記録する。AuditLog作成に失敗してもBean非公開処理はrollbackしない                 |
| <code>AdminArticleUsecase.CreateArticle</code> | <code>Create(article_create)</code>         | AdminによるArticle作成操作を記録する。AuditLog作成に失敗してもArticle作成はrollbackしない               |
| <code>AdminArticleUsecase.UpdateArticle</code> | <code>Create(article_update)</code>         | AdminによるArticle更新操作を記録する。AuditLog作成に失敗してもArticle更新はrollbackしない               |

| usecase                                           | 呼び出し                                    | 何をしているのか                                                           |
|---------------------------------------------------|-----------------------------------------|--------------------------------------------------------------------|
| <code>AdminArticleUsecase.PublishArticle</code>   | <code>Create(article_publish)</code>    | AdminによるArticle公開操作を記録する。AuditLog作成に失敗してもArticle公開処理はrollbackしない   |
| <code>AdminArticleUsecase.UnpublishArticle</code> | <code>Create(article_unpublish)</code>  | AdminによるArticle非公開操作を記録する。AuditLog作成に失敗してもArticle非公開処理はrollbackしない |
| <code>RankingBatchUsecase.Recalculate</code>      | <code>Create(ranking_batch_run)</code>  | ランキングバッチ実行を監査ログに記録する。AuditLog作成に失敗してもBatch本体はrollbackしない           |
| <code>InterestBatchUsecase.Recalculate</code>     | <code>Create(interest_batch_run)</code> | 興味プロフィール再計算バッチ実行を監査ログに記録する。AuditLog作成に失敗してもBatch本体はrollbackしない     |
| <code>AdminAuditUsecase.List</code>               | <code>List</code>                       | Admin画面で監査ログを検索・表示する                                               |

## Redis Repository詳細RateLimitRepository

| メソッド               | 引数                                                                       | 戻り値                                 | 用途                     |
|--------------------|--------------------------------------------------------------------------|-------------------------------------|------------------------|
| <code>Take</code>  | <code>key string, capacity int, refillRate float64, now time.Time</code> | <code>RateLimitResult, error</code> | Token Bucketで通過可否を判定する |
| <code>Reset</code> | <code>key string</code>                                                  | <code>error</code>                  | RateLimit状態を削除する       |

### Redis実装条件

| メソッド               | Redis処理                                   | 何をしているのか                           |
|--------------------|-------------------------------------------|------------------------------------|
| <code>Take</code>  | Lua scriptで現在token数、最終更新時刻を読み、補充、消費、保存を行う | Token Bucket方式で、対象リクエストを通してよいか判定する |
| <code>Reset</code> | <code>DEL key</code>                      | 指定keyのRateLimit状態を削除する             |

### usecaseからの呼び出し

| usecase / middleware                     | 呼び出し               | 何をしているのか                       |
|------------------------------------------|--------------------|--------------------------------|
| <code>RateLimitMiddleware</code>         | <code>Take</code>  | APIリクエストごとに制限を超えていないか確認する      |
| <code>AdminRateLimitUsecase.Reset</code> | <code>Reset</code> | 管理者操作で特定keyのRateLimit状態をリセットする |

## EventDedupRepository

| メソッド                        | 引数                                         | 戻り値                      | 用途                   |
|-----------------------------|--------------------------------------------|--------------------------|----------------------|
| <code>SetIfNotExists</code> | <code>key string, ttl time.Duration</code> | <code>bool, error</code> | 重複イベントでなければ保存する      |
| <code>Exists</code>         | <code>key string</code>                    | <code>bool, error</code> | 重複判定する               |
| <code>Delete</code>         | <code>key string</code>                    | <code>error</code>       | 必要に応じてdedup keyを削除する |

### Redis実装条件

| メソッド                        | Redis処理                          | 何をしているのか                        |
|-----------------------------|----------------------------------|---------------------------------|
| <code>SetIfNotExists</code> | <code>SET key 1 NX EX ttl</code> | keyが存在しない場合だけ保存し、TTLを付けて重複送信を防ぐ |
| <code>Exists</code>         | <code>EXISTS key</code>          | 指定keyがRedisに存在するか確認する           |
| <code>Delete</code>         | <code>DEL key</code>             | 指定keyをRedisから削除する               |

### dedup keyの例

| EventType                 | key例                                                                        | TTL |
|---------------------------|-----------------------------------------------------------------------------|-----|
| <code>impression</code>   | <code>dedup:impression:{actor}:{rankTargetID}:{placement}:{pagePath}</code> | 24h |
| <code>content_view</code> | <code>dedup:content_view:{actor}:{rankTargetID}:{pagePath}</code>           | 30m |

|                  |                                            |     |
|------------------|--------------------------------------------|-----|
| modal_impression | dedup:modal_impression:{modalDisplayLogID} | 24h |
|------------------|--------------------------------------------|-----|

## usecaseからの呼び出し

| usecase                        | 呼び出し                                                    | 何をしているのか                       |
|--------------------------------|---------------------------------------------------------|--------------------------------|
| EventUsecase.RecordImpression  | SetIfNotExists → trueなら<br>ActionEventRepository.Create | 重複していないimpressionだけDBに保存する     |
| EventUsecase.RecordContentView | SetIfNotExists → trueなら<br>ActionEventRepository.Create | 短時間で同じcontent_viewが重複記録されるのを防ぐ |
| ModalUsecase.ShowModal         | SetIfNotExists → trueなら modal_impression 記録             | 同じモーダル表示イベントを二重に記録しないようにする     |

## ModalSuppressionRepository

| メソッド                  | 引数                                                      | 戻り値          | 用途              |
|-----------------------|---------------------------------------------------------|--------------|-----------------|
| SetShown              | actorKey string, rankTargetID uint64, ttl time.Duration | error        | 同一候補の再表示を抑制する   |
| WasShown              | actorKey string, rankTargetID uint64                    | bool, error  | 同一候補が表示済みか確認する  |
| IncrementPageCount    | actorKey string, pagePath string, ttl time.Duration     | int64, error | 同一ページ表示回数を増やす   |
| IncrementSessionCount | actorKey string, ttl time.Duration                      | int64, error | 同一セッション表示回数を増やす |
| SetClosed             | actorKey string, rankTargetID uint64, ttl time.Duration | error        | 閉じた候補を抑制する      |
| WasClosed             | actorKey string, rankTargetID uint64                    | bool, error  | 閉じた候補か確認する      |

## Redis実装条件

| メソッド                  | Redis処理                                          | 何をしているのか                             |
|-----------------------|--------------------------------------------------|--------------------------------------|
| SetShown              | SET modal:shown:{actor}:{rankTargetID} 1 EX ttl  | 同じUserまたはGuestに、同じ候補を一定期間再表示しないようにする |
| WasShown              | EXISTS modal:shown:{actor}:{rankTargetID}        | 同じ候補を最近表示済みか確認する                     |
| IncrementPageCount    | INCR modal:page:{actor}:{pagePath} + 初回だけ EXPIRE | 同一ページ内のモーダル表示回数を増やし、表示上限を判定する        |
| IncrementSessionCount | INCR modal:session:{actor} + 初回だけ EXPIRE         | 同一セッション内のモーダル表示回数を増やし、表示上限を判定する      |
| SetClosed             | SET modal:closed:{actor}:{rankTargetID} 1 EX ttl | 閉じられた候補を一定期間再表示しないようにする              |
| WasClosed             | EXISTS modal:closed:{actor}:{rankTargetID}       | 同じ候補が最近閉じられたか確認する                    |

## usecaseからの呼び出し

| usecase                 | 呼び出し                                                              | 何をしているのか                                      |
|-------------------------|-------------------------------------------------------------------|-----------------------------------------------|
| ModalUsecase.CanShow    | WasShown / WasClosed / IncrementPageCount / IncrementSessionCount | 表示済み、閉じた候補、ページ内回数、セッション内回数を確認して、モーダル表示可否を判断する |
| ModalUsecase.ShowModal  | 表示成功後 SetShown                                                    | 表示した候補を一定期間再表示しないように記録する                      |
| ModalUsecase.CloseModal | 閉じた後 SetClosed                                                    | 閉じられた候補を一定期間再表示しないように記録する                     |

## BatchLockRepository

| メソッド    | 引数                                          | 戻り値         | 用途                 |
|---------|---------------------------------------------|-------------|--------------------|
| Acquire | key string, owner string, ttl time.Duration | bool, error | バッチロックを取得する        |
| Release | key string, owner string                    | error       | owner一致時のみロックを解放する |
| Extend  | key string, owner string, ttl time.Duration | bool, error | 長時間バッチのロック期限を延長する  |

## Redis実装条件

| メソッド    | Redis処理                      | 何をしているのか                     |
|---------|------------------------------|------------------------------|
| Acquire | SET key owner NX EX ttl      | バッチ開始前にロックを取得し、同じバッチの二重実行を防ぐ |
| Release | Lua scriptでowner一致時のみ DEL    | 自分が取得したロックだけを安全に解放する         |
| Extend  | Lua scriptでowner一致時のみ EXPIRE | 長時間バッチ中にロック期限を延長する           |

## usecaseからの呼び出し

| usecase                           | 呼び出し                   | 何をしているのか                                   |
|-----------------------------------|------------------------|--------------------------------------------|
| RankingBatchUsecase.Recalculate   | Acquire → 処理 → Release | Redis Lockを取得して、ランキングバッチが二重実行されないようにする     |
| AdminBatchUsecase.RunRankingBatch | Acquire → 処理 → Release | 手動実行時もRedis Lockを使い、自動実行や他の手動実行と衝突しないようにする |

## Filter / Aggregate DTO

Repository Interfaceで使う補助型。

### BeanSearchFilter

| フィールド      | Go型                | 用途                       |
|------------|--------------------|--------------------------|
| Keyword    | *string            | 名前、説明、FlavorNote検索       |
| Origin     | *string            | 産地                       |
| RoastLevel | *entity.RoastLevel | 焙煎度                      |
| Acidity    | *int               | 酸味                       |
| Bitterness | *int               | 苦味                       |
| Flavor     | *int               | 風味                       |
| Aroma      | *int               | 香り                       |
| Body       | *int               | ボディ                      |
| Sort       | string             | score / newest / popular |
| Limit      | int                | 件数                       |
| Offset     | int                | 開始位置                     |

### ArticleSearchFilter

| フィールド    | Go型     | 用途                       |
|----------|---------|--------------------------|
| Keyword  | *string | タイトル、要約、本文検索             |
| Category | *string | 記事カテゴリ                   |
| Sort     | string  | score / newest / popular |
| Limit    | int     | 件数                       |
| Offset   | int     | 開始位置                     |

### ContentMetricAggregate

| フィールド            | Go型    | 用途    |
|------------------|--------|-------|
| RankTargetID     | uint64 | 集計対象  |
| ImpressionCount  | int64  | 表示数   |
| ContentViewCount | int64  | 詳細閲覧数 |
| ClickCount       | int64  | クリック数 |



| フィールド                | Go型   | 用途        |
|----------------------|-------|-----------|
| StayTotalMs          | int64 | 総滞在時間     |
| SaveCount            | int64 | 保存数       |
| RatingCount          | int64 | 評価数       |
| GoodCount            | int64 | Good数     |
| BadCount             | int64 | Bad数      |
| ModalImpressionCount | int64 | モーダル表示数   |
| ModalClickCount      | int64 | モーダルクリック数 |
| ModalCloseCount      | int64 | モーダル閉じ数   |

注意:

ReSearchCount は持たない。

re\_search は検索条件変更イベントであり、RankTarget単位のcontent\_metricsには集計しない。

## RatingAggregate

| フィールド       | Go型     | 用途         |
|-------------|---------|------------|
| RatingCount | int64   | 評価数        |
| GoodCount   | int64   | Good数      |
| BadCount    | int64   | Bad数       |
| RatingAvg   | float64 | Good/Bad平均 |
| GoodRate    | float64 | Good率      |
| BadRate     | float64 | Bad率       |

## InterestAggregate

| フィールド          | Go型                      | 用途       |
|----------------|--------------------------|----------|
| UserID         | *uint64                  | Userの場合  |
| GuestSessionID | *uint64                  | Guestの場合 |
| Dimension      | entity.InterestDimension | 興味軸      |
| Value          | string                   | 興味軸の値    |
| ScoreDelta     | float64                  | 加算・減算値   |
| LastEventAt    | time.Time                | 最終イベント日時 |

## AuditLogFilter

| フィールド       | Go型                    | 用途    |
|-------------|------------------------|-------|
| ActorType   | *entity.AuditActorType | 操作者種別 |
| ActorUserID | *uint64                | 操作者ID |
| Action      | *entity.AuditAction    | 操作種別  |
| TargetType  | *string                | 対象種別  |
| TargetID    | *uint64                | 対象ID  |
| From        | *time.Time             | 開始日時  |
| To          | *time.Time             | 終了日時  |
| Limit       | int                    | 件数    |
| Offset      | int                    | 開始位置  |